Route Navigation using A* — Project Report

Foundations of Artificial Intelligence (25ML35T) | Innovative Assignment 1
Student: Dhilfa A | Register No: VH15152 | Batch 7 | Year/Sem/Sec: II / III / A

---

1. Problem Formulation

The task is to find the lowest-cost route between two locations on the Vel Tech campus, modelled as a weighted graph, and solve it using the A* search algorithm.

• **States:** a campus location, e.g. MainGate, Library, Hostel, CSEBlock (10 locations in total).

• **Initial state:** the user-specified start location.

• **Actions:** move along a road to a directly connected neighbouring location.

• **Goal test:** current location equals the requested destination.

• **Path cost:** sum of the road distances (edge weights) travelled from the start to the current node.

2. Approach

A* was implemented from first principles using a priority queue (min-heap) ordered by f(n) = g(n) + h(n), where g(n) is the actual cumulative road distance travelled from the start to node n, and h(n) is the straight-line (Euclidean) distance from n to the goal, computed from each location's (x, y) coordinates on the campus layout.

The heuristic is **admissible**: a straight line between two points can never be longer than any actual road path between them, so h(n) never overestimates the true remaining cost. An admissible heuristic guarantees that A* returns the optimal (minimum-cost) path — exactly like Dijkstra's algorithm — while typically expanding far fewer nodes, because the heuristic actively steers the search toward the goal instead of expanding outward uniformly in every direction.

A plain **BFS** (breadth-first search, ignoring edge weights and counting hops only) was also implemented purely as a baseline, to produce the required nodes-expanded comparison; BFS has no notion of a heuristic or edge cost, so it is not expected to be cost-optimal on a weighted graph.

Personalisation: edge weights are generated with `random.seed(15152)` (derived from register number VH15152), with a small random jitter applied to each base distance in `src/graph.py`, so this graph instance is unique to this submission.

3. Complexity

Let V be the number of locations (nodes) and E the number of roads (edges) in the campus graph. In this project V = 10 and E = 14.

• **A* time complexity:** O(E log V) in the practical case with a good heuristic and a binary-heap priority queue, up to O(b^d) in the worst case (b = branching factor, d = solution depth) if the heuristic gives little guidance — bounded here since the graph is small and finite.

• **A* space complexity:** O(V), to store the open set, the best-known costs, and the reconstructed paths.

• **BFS time/space complexity:** O(V + E) time, O(V) space.

Practical limit hit: at this graph size (10 nodes) the wall-clock time difference between A* and BFS is negligible — both run instantly. The complexity advantage of A* only shows up in the nodes-expanded metric, which is why that metric is used below instead of runtime, matching how the two algorithms

would diverge in practice on a much larger campus map or city graph.

4. Results

The table below compares nodes expanded by A* versus BFS across five start/goal pairs, run on the same seeded graph (register number VH15152). All figures were reproduced directly from `src/main.py`.

Start	Goal	A* path cost	A* nodes expanded	BFS nodes expanded
MainGate	Hostel	9.94	7	10
MainGate	SportsGround	7.48	5	7
Parking	Hostel	8.07	5	10
CSEBlock	Library	4.48	3	7
AdminBlock	Hostel	7.88	4	10

Across all five pairs, A* consistently expanded fewer nodes than BFS — on average about 44% fewer — because the Euclidean heuristic steers expansion toward nodes that are geometrically closer to the goal, rather than exploring every neighbour equally as BFS does.

Sample run (MainGate → Hostel):

```
$ python src/main.py --start MainGate --goal Hostel

--- A* Search ---
Path: MainGate -> Parking -> Auditorium -> Canteen -> Library -> Hostel
Total cost: 9.94
Nodes expanded: 7

--- BFS (baseline, ignores weights) ---
Path: MainGate -> AdminBlock -> CSEBlock -> AIMLBlock -> Hostel
Nodes expanded: 10

Summary: A* expanded 7 nodes vs BFS's 10 nodes to reach Hostel from MainGate.
```

Note that A*'s path (via Parking / Auditorium / Canteen / Library) uses more hops than the BFS path, yet has a lower total road distance — this is expected, since BFS minimises hop count while A* minimises actual travelled distance. Only A* is optimal with respect to the real edge weights.

5. Reflection

What was hard: designing coordinates for the campus locations so the Euclidean heuristic stayed genuinely admissible (never exceeding the true road distance) while still keeping the graph realistic took a few iterations. Getting the priority-queue tie-breaking right in the A* open list (so paths of equal f-cost don't cause errors when comparing list/tuple objects in the heap) also needed care.

What I'd improve: with more time I would pull real GPS coordinates for the actual campus layout instead of illustrative plan coordinates, and add a visualisation of the explored node set overlaid on the map, to make the A* vs BFS difference visually obvious rather than only numeric.

Honesty note: An AI assistant (Claude) was consulted for repository scaffolding, code structure suggestions, and report formatting. The algorithm design, the graph, and all code were reviewed and understood by me, and I can explain and modify every part in the viva.